MODULE C13 · FRONT-END · PARTIE 1

HTML et CSS

Structurer et mettre en forme une page web, de la maquette au navigateur

PAR HAROUNA KANE – Groupe SR3
SIMPLON

OBJECTIFS

Ce que vous saurez faire

- Structurer une page en **HTML sémantique**
- Mettre en forme avec **CSS** : sélecteurs, boîte, typographie
- Mettre en page avec **Flexbox** et **Grid**
- Rendre une page **responsive** et **accessible**

Compétence visée : **C13**, développer le front-end d'une application à partir de maquettes et du parcours utilisateur.

PROGRAMME

Une page qui se construit, étape par étape

Thème	Production
Le web, HTML, balises sémantiques, tableaux, formulaires	Exercice 1 : la page d'une formation
CSS, cascade, modèle de boîte, Flexbox, Grid	Exercice 2 : styliser la page
Responsive, accessibilité, vérification	TP : le catalogue des formations

À chaque étape : des apports courts, des démonstrations à reproduire, puis un exercice. On code dès le début.

FIL ROUGE

Le catalogue des formations du centre

Les données sont celles de la base que vous avez conçue : formations, sessions, niveaux, durées. La même page va évoluer tout au long du module.

01

HTML et CSS

Une page statique, fidèle à la maquette, sur ordinateur et sur mobile

Ce module

02

JavaScript

Le catalogue devient dynamique et se charge depuis une API

03

React

La même application, découpée en composants

PARTIE 1

Le web et HTML

Comment une page arrive dans le navigateur, et comment on la structure

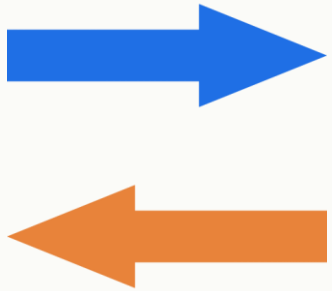
LE WEB

Comment une page arrive à l'écran

Navigateur

le client

1. Requête HTTP : « donne-moi catalogue.html »



2. Réponse : HTML, puis CSS, JS et images

Serveur

héberge les fichiers

3. Le navigateur lit le HTML, construit le **DOM** (l'arbre des éléments), applique le CSS, puis affiche la page.

LE WEB

Trois langages, trois rôles

HTML

La structure

Ce qu'est le contenu : un titre, un paragraphe, une liste, un formulaire

Les murs de la maison

CSS

La présentation

À quoi il ressemble : couleurs, tailles, disposition, écrans

La peinture et l'aménagement

JavaScript

Le comportement

Ce qu'il fait : réagir aux clics, charger des données

L'électricité · partie 2

OUTIL

Les DevTools : votre meilleur allié

Ouvrir : **clic droit › Inspecter** ou **F12**

Éléments

L'arbre HTML de la page, modifiable en direct

Styles

Les règles CSS appliquées, et celles qui sont écrasées

Réseau

Chaque fichier demandé au serveur, son poids, son temps

À vous : ouvrez un site connu et changez le texte de son titre principal.

HTML

Le squelette d'une page

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport"
      content="width=device-width, initial-scale=1">
    <title>Catalogue des formations</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Nos formations</h1>
  </body>
</html>
```

DOCTYPE

« Ceci est du HTML5 »

lang="fr"

Langue de la page, utile aux lecteurs d'écran

head

Informations sur la page, invisibles à l'écran

body

Tout ce qui s'affiche

HTML

Anatomie d'un élément

```
<a href="formation.html">Voir la formation</a>
```

Balise ouvrante

<a ...>

Attribut

nom="valeur", une précision

Contenu

Ce que voit l'utilisateur

Balise fermante

, avec le /

Orphelines : pas de contenu, pas de fermeture. <img>,
, <input>, <meta>

Imbrication : on ferme dans l'ordre inverse. ...

HTML

Titres, paragraphes et listes

```
<h1>Développeur IA</h1>

<p>Une formation de

 <strong>18 mois</strong>.</p>


<h2>Programme</h2>

<ul>

 <li>Bases de données</li>

 <li>Front-end</li>

 <li>Intelligence artificielle</li>

</ul>

<!-- ol pour une liste numérotée -->
```

- **Un seul h1** par page : le sujet principal
- **Ne pas sauter de niveau** : h2 après h1, h3 après h2
- **strong** et **em** donnent du sens (important, accentué), pas seulement du gras
- On choisit une balise pour **ce que le contenu est**, jamais pour sa taille

HTML

Liens et images

```
<!-- lien absolu : un autre site -->
<a href="https://simplon.co">Simplon</a>
<!-- lien relatif : une page du projet -->
<a href="pages/sessions.html">Les sessions</a>
<!-- ancre : un endroit de la page -->
<a href="#contact">Nous contacter</a>

<img src="img/dev-ia.jpg" alt="Apprenants en atelier de code">
```

alt est obligatoire

Il décrit l'image à qui ne la voit pas : lecteur d'écran, image non chargée.

Image décorative ?

alt="" (vide) : le lecteur d'écran l'ignore. Ne jamais omettre l'attribut.

EXERCICE 1 · INDIVIDUEL

La page d'une formation

1. Créez index.html avec le squelette HTML5

2. Un titre, une description, la durée et le niveau

3. Le programme sous forme de liste

4. Un lien vers une page « sessions » et une image avec alt

5. Poussez le tout sur un dépôt GitHub

Critères de réussite

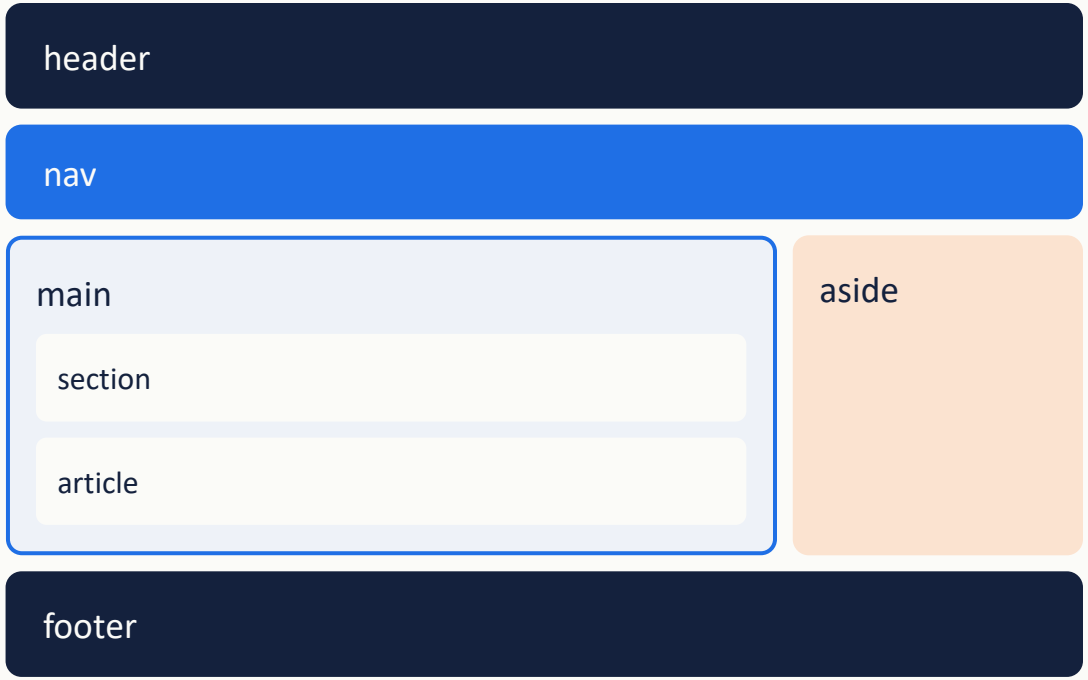
Aucune erreur au validateur W3C

Une balise adaptée à chaque contenu

Pas de CSS pour l'instant

SÉMANTIQUE

Les zones d'une page



header : logo, titre du site

nav : les liens de navigation principaux

main : le contenu propre à la page, une seule fois

section : un regroupement thématique, avec un titre

article : un contenu autonome, comme une carte formation

aside : un contenu annexe

footer : contact, mentions légales

SÉMANTIQUE

Pourquoi pas que des div ?

Accessibilité

Un lecteur d'écran peut sauter directement au menu ou au contenu principal

Référencement

Les moteurs de recherche comprennent ce qui est important

Lisibilité

Le code se relit facilement, en équipe et six mois plus tard

Une **div** ne dit rien de son contenu. Une **nav** dit « je suis le menu ».

HTML

Les tableaux : pour des données

```
<table>
  <caption>Prochaines sessions</caption>
  <tr>
    <th>Ville</th>
    <th>Début</th>
    <th>Places</th>
  </tr>
  <tr>
    <td>Creil</td>
    <td>06/10/2026</td>
    <td>16</td>
  </tr>
</table>
```

Prochaines sessions

Ville	Début	Places
Creil	06/10/2026	16
Paris	04/01/2027	20

caption : le titre du tableau · **th** : une en-tête · **td** : une cellule

Un tableau sert aux données, jamais à la mise en page.
Pour disposer des blocs : Flexbox et Grid.

HTML

Les formulaires

```
<form action="/inscription" method="post">
  <label for="email">Email</label>
  <input type="email" id="email"
         name="email" required>

  <label for="session">Session</label>
  <select id="session" name="session">
    <option>Creil · octobre</option>
    <option>Paris · janvier</option>
  </select>

  <button type="submit">S'inscrire</button>
</form>
```

label for = **id** de l'input : cliquer sur le texte active le champ, et le lecteur d'écran le lit

type : email, number, date, password… le navigateur adapte le clavier et vérifie

name : le nom de la donnée envoyée au serveur

required : validation sans une ligne de JavaScript

PARTIE 2

CSS

Habiller la page : sélecteurs, cascade, boîtes, Flexbox et Grid

CSS

Où écrire le CSS, et comment

Attribut style

<p style="color:red">

À éviter : mélange tout

Balise style

<style>...</style>

Pour un test rapide

Fichier externe

<link href="style.css">

La bonne pratique

```
h1 {
  color: #1F6FE5;
  font-size: 2.5rem;
}
```

Sélecteur : qui est concerné

Propriété : ce qu'on change

Valeur : comment, terminée par ;

CSS

Les sélecteurs essentiels

Sélecteur	Cible	Exemple
p	Tous les éléments p	Le texte courant
.carte	Les éléments class="carte"	Chaque carte formation
#contact	L'élément id="contact", unique	Une ancre
nav a	Les a situés dans une nav	Les liens du menu
h1, h2	Les h1 et les h2	Une police commune
a:hover	Un lien survolé	Un changement de couleur

Le réflexe : on stylise avec des **classes**. Les id servent aux ancres et à JavaScript.

CSS

La cascade : qui gagne ?

1. La spécificité
id > classe > élément

2. L'ordre
À égalité, la dernière règle écrite gagne

3. L'héritage
Couleur et police passent du parent aux enfants

```
<p class="prix" id="promo">890 €</p>

p { color: black; }
.prix { color: blue; }
#promo { color: red; }
```

Résultat : **rouge**, l'id est le plus spécifique.

CSS

Le modèle de boîte




```
* { box-sizing: border-box; }

.carte {
  width: 320px;
  padding: 24px;
  border: 1px solid #ddd;
  margin: 16px;
}
```

border-box : width inclut padding et bordure. La carte fait vraiment 320 px.

CSS

Unités, couleurs, typographie

Unité	Usage
px	Bordures, petits détails
rem	Textes et espacements (1rem = 16px)
%	Largeurs relatives au parent
vw / vh	Fraction de l'écran

Couleurs : red · #1F6FE5 · rgb(31, 111, 229)

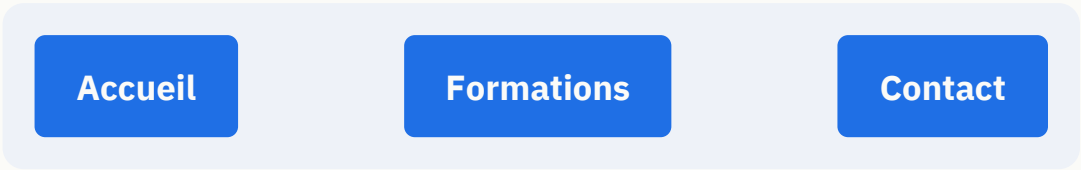
```
body {
  font-family: "Inter", Arial,
    sans-serif;
  font-size: 1rem;
  line-height: 1.5;
  color: #14213D;
}
```

On prévoit toujours une police de **secours**, et une hauteur de ligne d'environ 1,5 pour la lecture.

MISE EN PAGE

Flexbox : aligner sur une ligne

```
nav ul {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
}
```



display: flex se met sur le **parent** ; les enfants s'alignent

flex-direction : row (ligne) ou column

justify-content : répartition sur l'axe principal

align-items : alignement sur l'axe secondaire

gap : l'espace entre les enfants

flex-wrap: wrap : passer à la ligne si besoin

MISE EN PAGE

Grid : une grille en lignes et colonnes

```
.catalogue {
  display: grid;
  grid-template-columns:
    repeat(3, 1fr);
  gap: 1.5rem;
}
```

1fr = une part de l'espace disponible. Trois colonnes égales, quelle que soit la largeur.

Les cartes passent toutes seules à la ligne suivante.



3 colonnes de 1fr, espacées par gap

MISE EN PAGE

Flexbox ou Grid ?

Flexbox

Une dimension

Une ligne ou une colonne d'éléments : un menu, des boutons, le contenu d'une carte

Grid

Deux dimensions

Des lignes et des colonnes en même temps : une grille de cartes, la structure de la page

On les combine : **Grid pour la page**, **Flexbox dans chaque carte**.

EXERCICE 2 · EN BINÔMES

Styliser la page de la formation

1. Reliez un fichier style.css à votre page de l'exercice 1

2. Appliquez la charte : deux couleurs, une police, des espacements en rem

3. Mettez la navigation et le contenu des cartes en **Flexbox**

4. Disposez les cartes des formations en **Grid**

5. Correction collective

Critères de réussite

Aucun attribut style dans le HTML

Des classes nommées par leur rôle

Un commit à chaque étape

PARTIE 3

Responsive, accessibilité et TP

Une page qui fonctionne sur tous les écrans et pour tout le monde

RESPONSIVE

Mobile first et media queries

```
/* Mobile : par défaut */
.catalogue {
  display: grid;
  grid-template-columns: 1fr;
}

/* Tablette */
@media (min-width: 768px) {
  .catalogue { grid-template-columns: repeat(2, 1fr); }
}

/* Ordinateur */
@media (min-width: 1200px) {
  .catalogue { grid-template-columns: repeat(3, 1fr); }
}
```

Mobile first : on écrit d'abord pour le petit écran, puis on enrichit avec min-width

Sans la balise meta viewport, le mobile affiche la version bureau en miniature

Tester : DevTools › mode appareil (Ctrl+Maj+M), de 375 px à 1440 px

BONNES PRATIQUES

Variables CSS et images fluides

```
:root {
  --bleu: #1F6FE5;
  --texte: #14213D;
  --espace: 1.5rem;
}

.bouton {
  background: var(--bleu);
  padding: var(--espace);
}
```

La charte change ? **Une seule ligne** à modifier.

```
img {
  max-width: 100%;
  height: auto;
}
```

L'image ne dépasse jamais de son conteneur et garde ses proportions.

Ordre du fichier : variables, base, mise en page, composants, media queries

ACCESSIBILITÉ

Six réflexes pour une page accessible

- ✓ Un **alt** pertinent sur chaque image
- ✓ Un **label** relié à chaque champ
- ✓ Un **contraste** d'au moins 4,5:1
- ✓ Tout utilisable **au clavier** (Tab, Entrée)
- ✓ Des **titres hiérarchisés** h1 › h2 › h3
- ✓ Des **balises sémantiques** plutôt que des div

ARIA en dernier recours : une balise HTML native fait toujours mieux qu'un attribut ajouté.

QUALITÉ

Vérifier son travail

Validateur W3C

validator.w3.org : balises mal fermées, attributs manquants

DevTools

Règles écrasées, débordements, rendu mobile

Lighthouse

Un score d'accessibilité, de performance et de bonnes pratiques

Objectif du TP : **0 erreur W3C** et un score d'accessibilité Lighthouse **au-dessus de 90**.

TP · EN BINÔMES

Le catalogue des formations

Reproduisez la maquette fournie, versions bureau et mobile, avec les données de la base du centre.

1. En-tête et navigation
2. Grille de cartes des formations
3. Tableau des prochaines sessions
4. Formulaire de demande d'information
5. Affichage correct sur mobile et sur ordinateur

Livrable

Un dépôt GitHub par binôme, avec un README

Des commits réguliers des deux membres

Lien du dépôt déposé sur Discord

ÉVALUATION

Les critères du TP

Critère	Ce qui est regardé
Maquette	La page est fidèle à la maquette fournie
HTML	Valide au W3C, balises sémantiques adaptées
CSS	Fichier externe organisé, classes, variables
Responsive	Lisible de 375 px à 1440 px, sans défilement horizontal
Accessibilité	alt, label, contraste, navigation au clavier
Git	Commits réguliers et messages clairs, des deux membres

SYNTHÈSE

Ce qu'il faut retenir

1. HTML dit **ce qu'est** le contenu, CSS dit **à quoi il ressemble**
2. On choisit la balise pour son **sens**, jamais pour son apparence
3. On stylise avec des **classes**, dans un fichier externe
4. **Flexbox** pour une ligne, **Grid** pour une grille
5. On pense **mobile d'abord**, puis on élargit
6. L'accessibilité se construit **dès la première ligne**

ET ENSUITE

JavaScript

Le DOM
réagir aux clics

[icon: ArrowRight]

fetch et API
charger le catalogue

[icon: ArrowRight]

React
en composants

Gardez votre catalogue : c'est lui qui va prendre vie.